HW10

Nathan Krieger

2026-04-17

Problem 1: Weekly Dataset

We will use the Weekly data set from the ISLR2 library. It contains 1,089 weekly returns for 21 years, from the beginning of 1990 to 2010.

(a)

Use the full data set to perform a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Use the summary function to report your results.

```r
library(ISLR2)

head(Weekly)
```

```
##   Year   Lag1   Lag2   Lag3   Lag4   Lag5    Volume  Today Direction
## 1 1990  0.816  1.572 -3.936 -0.229 -3.484 0.1549760 -0.270      Down
## 2 1990 -0.270  0.816  1.572 -3.936 -0.229 0.1485740 -2.576      Down
## 3 1990 -2.576 -0.270  0.816  1.572 -3.936 0.1598375  3.514        Up
## 4 1990  3.514 -2.576 -0.270  0.816  1.572 0.1616300  0.712        Up
## 5 1990  0.712  3.514 -2.576 -0.270  0.816 0.1537280  1.178        Up
## 6 1990  1.178  0.712  3.514 -2.576 -0.270 0.1544440 -1.372      Down
```

```r
glm_fit_full <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
                    data = Weekly,
                    family = "binomial")

summary(glm_fit_full)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = "binomial", data = Weekly)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.26686    0.08593   3.106   0.0019 **
## Lag1        -0.04127    0.02641  -1.563   0.1181
## Lag2         0.05844    0.02686   2.175   0.0296 *
## Lag3        -0.01606    0.02666  -0.602   0.5469
## Lag4        -0.02779    0.02646  -1.050   0.2937
## Lag5        -0.01447    0.02638  -0.549   0.5833
## Volume      -0.02274    0.03690  -0.616   0.5377
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1496.2  on 1088  degrees of freedom
## Residual deviance: 1486.4  on 1082  degrees of freedom
## AIC: 1500.4
##
## Number of Fisher Scoring iterations: 4
```

(b)

Set a threshold of 0.5. Compute the confusion matrix and overall correct classification rate. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

```r
glm_probs_full <- predict(glm_fit_full, type = "response")


glm_pred_full <- rep("Down", length(glm_probs_full))
glm_pred_full[glm_probs_full > 0.5] <- "Up"

table(glm_pred_full, Weekly$Direction)
```

```
##
## glm_pred_full Down  Up
##          Down   54  48
##          Up    430 557
```

```r
mean(glm_pred_full == Weekly$Direction)
```

```
## [1] 0.5610652
```

The overall correct classification rate is 56% so it's wrong 44% of the time. The confusion matrix shows predicted down but actually up being 430 is a very large of source of concern.

(c)

Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Set a threshold of 0.5. Compute the confusion matrix and overall correct classification rate on the test set (that is, data from 2009 and 2010).

```r
train <- Weekly[Weekly$Year <= 2008, ]
test <- Weekly[Weekly$Year > 2008, ]

glm_fit_split <- glm(Direction ~ Lag2, data = train, family = "binomial")

glm_probs_split <- predict(glm_fit_split, newdata = test, type = "response")

glm_pred_test <- rep("Down", length(glm_probs_split))
glm_pred_test[glm_probs_split > 0.5] <- "Up"

table(Predicted = glm_pred_test, Actual = test$Direction)
```

```
##          Actual
## Predicted Down Up
##      Down    9  5
##      Up     34 56
```

```r
mean(glm_pred_test == test$Direction)
```

```
## [1] 0.625
```

(d)

Repeat (c) using LDA.

```r
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:ISLR2':
##
##     Boston
```

```r
lda_fit <- lda(Direction ~ Lag2, data = train)
lda_pred <- predict(lda_fit, newdata = test)

table(Predicted = lda_pred$class, Actual = test$Direction)
```

```
##          Actual
## Predicted Down Up
##      Down    9  5
##      Up     34 56
```

```r
mean(lda_pred$class == test$Direction)
```

```
## [1] 0.625
```

(e)

Repeat (c) using QDA.

```r
qda_fit <- qda(Direction ~ Lag2, data = train)
qda_pred <- predict(qda_fit, newdata = test)

table(Predicted = qda_pred$class, Actual = test$Direction)
```

```
##          Actual
## Predicted Down Up
##      Down    0  0
##      Up     43 61
```

```r
mean(qda_pred$class == test$Direction)
```

```
## [1] 0.5865385
```

(f)

Repeat (c) using Naive Bayes.

```r
library(e1071)

nb_fit <- naiveBayes(Direction ~ Lag2, data = train)
nb_pred <- predict(nb_fit, newdata = test)

table(Predicted = nb_pred, Actual = test$Direction)
```

```
##          Actual
## Predicted Down Up
##      Down    0  0
##      Up     43 61
```

```r
mean(nb_pred == test$Direction)
```

```
## [1] 0.5865385
```

Problem 2: Bootstrap again

We will use a heart disease data set for this problem. To see information on the data and how to load it into R, check the heart.R file. You can download the data from Canvas or read the data from the website directly.

(a)

Fit a logistic regression model on the entire dataset with chd as the response and age as the only predictor. Based on the model you just fit, what is the predicted probability that Y = 1 given age = 50? We'll denote this as ^P (Y = 1 | age = 50). Report that probability here.

Note: make sure that age is a data.frame: x = data.frame(age = 50).

```r
heart = read.table('https://hastie.su.domains/ElemStatLearn/datasets/SAheart.data', sep=",",head=T,row.n

glm_heart <- glm(chd ~ age, data = heart, family = "binomial")

new_data <- data.frame(age = 50)
prob_age_50 <- predict(glm_heart, newdata = new_data, type = "response")

cat("Predicted probability P(Y=1 | age=50):", prob_age_50, "\n")
```

```
## Predicted probability P(Y=1 | age=50): 0.4215756
```

(b)

Bootstrap the standard error of ^P (Y = 1 | age = 50). Report the standard error here.

```r
library(boot)

prob_boot_fn <- function(data, index) {
  fit <- glm(chd ~ age, data = data, subset = index, family = "binomial")

  pred <- predict(fit, newdata = data.frame(age = 50), type = "response")

  return(pred)
}

set.seed(1)
boot_results <- boot(data = heart, statistic = prob_boot_fn, R = 2000)

se_boot_prob <- sd(boot_results$t)

print(boot_results)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
```

```
##
##
## Call:
## boot(data = heart, statistic = prob_boot_fn, R = 2000)
##
##
## Bootstrap Statistics :
##      original        bias    std. error
## t1* 0.4215756 -0.0002542733  0.02763202
```

```r
cat("Bootstrap SE of the predicted probability:", se_boot_prob, "\n")
```

```
## Bootstrap SE of the predicted probability: 0.02763202
```

Problem 3: Concept Review

Suppose you split a data set into a training set and a test set, both of which have a sample size of 2,500. For simplicity, let's assume that p = 1 (only one predictor) and the response is Y = 0 or Y = 1. We obtain the following estimates for the training set:

$\hat{}\ \mu\ 0 = 3.4$, $\hat{}\ \mu\ 1 = 5.1$, $\hat{}\ \sigma\ 2 = 4$, $\hat{}\ \pi\ 0 = 0.32$, $\hat{}\ \pi\ 1 = 0.68$, and for the test set: $\hat{}\ \mu\ 0 = 3.2$, $\hat{}\ \mu\ 1 = 5.5$, $\hat{}\ \sigma\ 2 = 4.5$, $\hat{}\ \pi\ 0 = 0.35$, $\hat{}\ \pi\ 1 = 0.65$.

We assume that the predictor X1 given the response follows a normal distribution and that the variance between the two classes is the same. Based on this information, write out the linear discriminant function for Y as a function of X1. Explain when a test observation will be assigned to Y = 0 and when it will be assigned to Y = 1.

Hint: If you find yourself needing to do some complicated math derivation, then stop.

Rethink the question. Your classification rule should involve real numbers from the estimates ($\hat{}\ \mu$, $\hat{}\ \sigma\ 2$, etc.) given above.

From class we have:

$\delta_k(x_1) = x_1 \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + log(\pi_k)$

So for Y = 0:

$\delta_0(x) = x\frac{3.4}{4} - \frac{3.4^2}{2*4} + log(0.32)$ $\delta_0(x) = 0.85x - 2.584$

For Y = 1:

$\delta_1(1) = x\frac{5.1}{4} - \frac{5.1^2}{2*4} + log(0.68)$

$\delta_1(x) = 1.275x - 3.637$

Set them equal to each other:

$1.275x - 3.637 = 0.85x - 2.584$

$x = 2.478$

If a test observation x is greater than 2.478 it will be assigned to Y = 1. If a test observation x is less than 2.478 it will be assigned to Y = 0.

Problem 4: MNIST handwritten digit database

Load the handwritten digits (MNIST) dataset into R using the R scripts we went over in class.

(a)

Randomly select 3000 observations from the training set and randomly select 100 observations from the test set. Implement KNN classification. Report the following:

- Carry out 10-fold cross-validation on the training set to determine the optimal K. Try K = 1, 5, 7, 9. What is the optimal K?
- Use this optimal K to implement KNN classification on the test set. Report your confusion matrix and misclassification error rate on the test set.

```
set.seed(1)

library(caret)
```

```
## Loading required package: ggplot2

##
## Attaching package: 'ggplot2'

## The following object is masked from 'package:e1071':
##
##     element

## Loading required package: lattice

##
## Attaching package: 'lattice'

## The following object is masked from 'package:boot':
##
##     melanoma
```

```
library(class)
#install.packages("dslabs")
library(dslabs)

# Reading in data
mnist <- read_mnist()

index_train <- sample(1:nrow(mnist$train$images), 3000)
index_test  <- sample(1:nrow(mnist$test$images), 100)

x_train <- mnist$train$images[index_train, ]
y_train <- as.factor(mnist$train$labels[index_train])

x_test  <- mnist$test$images[index_test, ]
y_test  <- as.factor(mnist$test$labels[index_test])

# see the picture for 5th observation
#i <- 5
#image(1:28, 1:28, matrix(mnist$test$images[i,], nrow=28)[ , 28:1],
#      col = gray(seq(0, 1, 0.05)), xlab = "", ylab="")
## the labels for this image is:
#mnist$test$labels[i]


#index.train = sample(1:dim(train$x)[1], 3000, replace = FALSE)
#index.test = sample(1:dim(test$x)[1], 100, replace = FALSE)
```

```r
#sample_x = train$x[index.train, ]
#sample_y = train$y[index.train]

# KNN CV
K = c(1, 3, 5, 7, 9)
fold <- 10

folds <- cut(seq(1, nrow(x_train)), breaks = fold, labels = FALSE)

cv_errors <- numeric(length(K))

for (i in 1:length(K)) {

  current_cv_error <- numeric(fold)

  k_val <- K[i]

  for (j in 1:fold) {

    val_indices <- which(folds == j)

    cv_train_x <- x_train[-val_indices, ]
    cv_train_y <- y_train[-val_indices]

    cv_val_x <- x_train[val_indices, ]
    cv_val_y <- y_train[val_indices]

    # Run KNN for this fold
    pred <- knn(train = cv_train_x, test = cv_val_x, cl = cv_train_y, k = k_val)

    current_cv_error[i] <- mean(pred != cv_val_y)
  }

  # Store the error rate for this fold
    cv_errors[i] <- mean(current_cv_error)
}


optimal_k <- K[which.min(cv_errors)]

cat("Optimal K based on 10-fold CV:", optimal_k, "\n")
```

```
## Optimal K based on 10-fold CV: 5
```

```r
final_pred <- knn(train = x_train, test = x_test, cl = y_train, k = optimal_k)

conf_matrix <- table(Predicted = final_pred, Actual = y_test)
print(conf_matrix)
```

```
##          Actual
## Predicted  0  1  2  3  4  5  6  7  8  9
##         0 10  0  0  0  0  0  0  0  0  1
##         1  0 10  0  0  0  0  0  1  0  0
##         2  0  0  4  0  0  0  0  1  0  0
```

```
##         3  0  0  0 10  0  1  0  0  0  0
##         4  0  0  0  0  5  0  0  0  0  0
##         5  0  0  0  2  0  7  0  0  0  0
##         6  1  0  0  0  0  0 12  0  0  0
##         7  0  0  0  0  0  0  0 12  0  2
##         8  0  0  1  0  0  0  0  0  9  0
##         9  0  0  0  0  1  0  0  0  0 10
```

```
test_error <- mean(final_pred != y_test)
cat("Test Error Rate with K =", optimal_k, ":", test_error, "\n")
```

```
## Test Error Rate with K = 5 : 0.11
```

(b)

Try to implement LDA on the MNIST dataset. What kind of error message do you obtain? Do some digging and explain what this error message means.

```
# (b) LDA attempt on MNIST
#lda_model <- lda(x = x_train, grouping = y_train)
```

I get an error that the pixels appear to be constant within groups. This is because there are certain spots of the image that are never a part of the number. For example, the corners of the image for the number 1. This means they have zero variance and provide no evidence for the model to weigh. Behind the scenes there is a division by zero that is happening.

(c)

Discuss how this dataset highlights some of the advantages of using KNN for classification.

KNN makes no assumptions on the distribution unlike LDA.

LDA tries to find one average which is a problem for different writing styles, KNN just looks for the closest neighbor.

KNN doesn't care if some pixels are constant.

Problem 5: Fashion MNIST (practice)

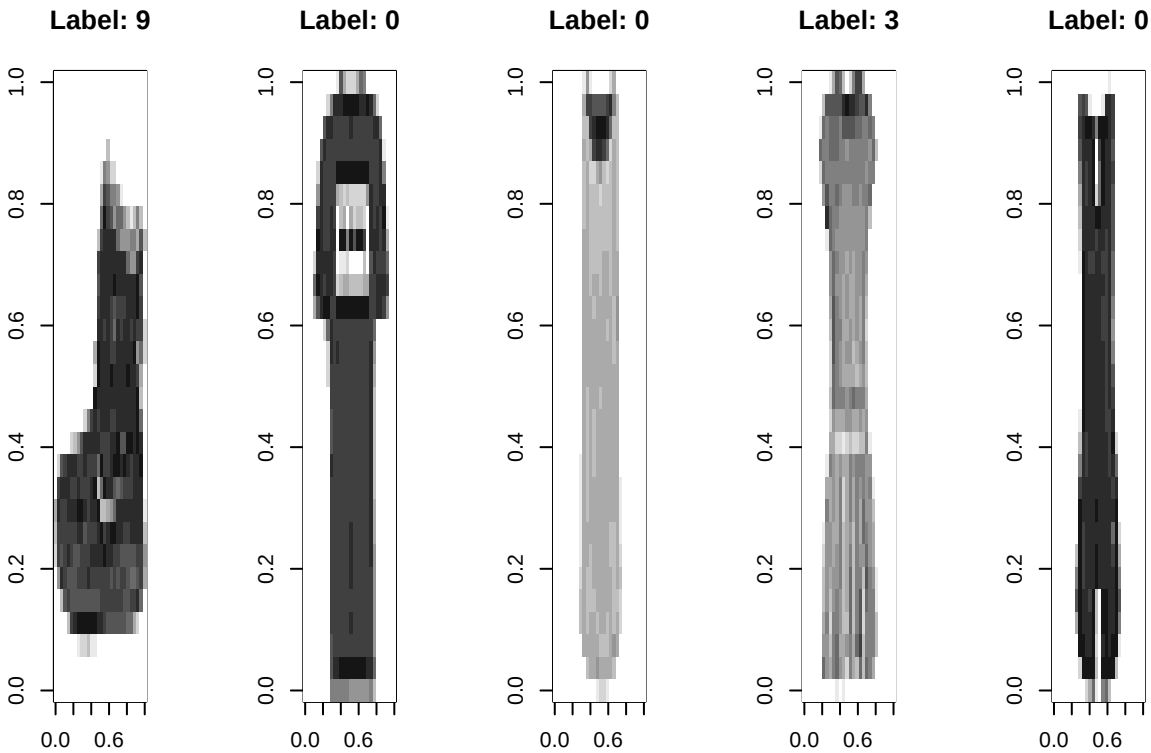
Many people consider the handwritten digits database to be far too easy for classifiers these days. More challenging datasets have appeared as new benchmarks. One example is the fashion MNIST dataset: https://github.com/zalandoresearch/fashion-mnist. Read up on the documentation for this dataset. Then, just like we did with the handwritten digits database, download the training and test set for this data. Load it into R using the same R scripts used in Problem 1.

(a)

Produce plots of the first 5 observations in the training set. What do you see?

```
library(caret)
library(class)


path = "~/git/DS301/HW10/mnist"

train_image_path = file.path(path, "train-images-idx3-ubyte")
test_image_path  = file.path(path, "t10k-images-idx3-ubyte")
```

```r
train_label_path = file.path(path, "train-labels-idx1-ubyte")
test_label_path  = file.path(path, "t10k-labels-idx1-ubyte")

load_mnist <- function() {
  load_image_file <- function(filename) {
    ret = list()
    f = file(filename, "rb")
    readBin(f, "integer", n = 1, size = 4, endian = "big")
    ret$n = readBin(f, "integer", n = 1, size = 4, endian = "big")
    nrow = readBin(f, "integer", n = 1, size = 4, endian = "big")
    ncol = readBin(f, "integer", n = 1, size = 4, endian = "big")
    x = readBin(f, "integer", n = ret$n * nrow * ncol, size = 1, signed = FALSE)
    ret$x = matrix(x, ncol = nrow * ncol, byrow = TRUE)
    close(f)
    ret
  }

  load_label_file <- function(filename) {
    f = file(filename, "rb")
    readBin(f, "integer", n = 1, size = 4, endian = "big")
    n = readBin(f, "integer", n = 1, size = 4, endian = "big")
    y = readBin(f, "integer", n = n, size = 1, signed = FALSE)
    close(f)
    y
  }

  train <<- load_image_file(train_image_path)
  test  <<- load_image_file(test_image_path)

  train$y <<- load_label_file(train_label_path)
  test$y  <<- load_label_file(test_label_path)
}


show_digit <- function(arr784, col = gray(12:1/12), ...) {
  image(matrix(arr784, nrow = 28)[, 28:1], col = col, ...)
}

mnist <- load_mnist()

# show the 5th image in the training set
# Produce plots of the first 5 observations
par(mfrow = c(1, 5)) # Set up the plot grid
for (i in 1:5) {
  show_digit(train$x[i, ], main = paste("Label:", train$y[i]))
}
```



This is a harder problem than the numbers because of the complicated, unique shapes. Also there is wider range of values where the numbers were pretty consistent.

(b)

Repeat the workflow from Problem 4(a) for this dataset. How do your confusion matrices and misclassification error rates compare?

```
mnist <- read_mnist()

index.train = sample(1:dim(train$x)[1], 3000, replace = FALSE)
index.test = sample(1:dim(test$x)[1], 100, replace = FALSE)

sample_x = train$x[index.train, ]
sample_y = train$y[index.train]

K = c(1, 3, 5, 7, 9)

fold <- 10

folds <- cut(seq(1, nrow(x_train)), breaks = fold, labels = FALSE)

cv_errors <- numeric(length(K))

for (i in 1:length(K)) {

  current_cv_error <- numeric(fold)

  k_val <- K[i]

  for (j in 1:fold) {
```

```r
    val_indices <- which(folds == j)

    cv_train_x <- sample_x[-val_indices, ]
    cv_train_y <- sample_y[-val_indices]

    cv_val_x <- sample_x[val_indices, ]
    cv_val_y <- sample_y[val_indices]

    # Run KNN for this fold
    pred <- knn(train = cv_train_x, test = cv_val_x, cl = cv_train_y, k = k_val)

    current_cv_error[i] <- mean(pred != cv_val_y)
  }

  # Store the error rate for this fold
    cv_errors[i] <- mean(current_cv_error)
}


optimal_k <- K[which.min(cv_errors)]

cat("Optimal K based on 10-fold CV:", optimal_k, "\n")
```

```
## Optimal K based on 10-fold CV: 7
```

```r
x_test <- test$x[index.test, ]
y_test <- test$y[index.test]

final_pred <- knn(train = sample_x, test = x_test, cl = sample_y, k = optimal_k)

conf_matrix <- table(Predicted = final_pred, Actual = y_test)
print(conf_matrix)
```

```
##          Actual
## Predicted  0  1  2  3  4  5  6  7  8  9
##         0  9  0  0  0  0  0  2  0  0  0
##         1  0 14  0  0  0  0  0  0  0  0
##         2  0  0  5  0  1  0  1  0  0  0
##         3  0  0  0  9  1  0  0  0  0  0
##         4  0  0  0  1 10  0  2  0  0  0
##         5  0  0  0  0  0  4  0  0  0  0
##         6  0  0  3  0  0  0  2  0  1  0
##         7  0  0  0  0  0  2  0 10  0  1
##         8  0  0  0  0  0  0  0  0  8  0
##         9  0  0  0  0  0  1  0  1  0 12
```

```r
test_error <- mean(final_pred != y_test)
cat("Test Error Rate with K =", optimal_k, ":", test_error, "\n")
```

```
## Test Error Rate with K = 7 : 0.17
```

The misclassification rate for clothes is much higher than for numbers. This makes sense because the clothes seems to be a much harder problem. The confusion matrix for fashion has a lot less zeros.